Alma Mater Studiorum-Università di Bologna
Scuola di Ingegneria

Interfacce standard
Comparatori & co.

Corso di Laurea in Ingegneria Informatica

Anno accademico 2025/2026

Prof. ENRICO DENTI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)

ALMA MATER STUDIORUM - UNIVERSITÀ DI BOLOGNA
IL PRESENTE MATERIALE È RISERVATO AL PERSONALE DELL'UNIVERSITÀ DI BOLOGNA E NON PUÒ ESSERE UTILIZZATO AI TERMINI DI LEGGE DA ALTRE PERSONE O PER FINI NON ISTITUZIONALI

INTERFACCE STANDARD

Esistono molte interfacce standard, destinate a esprimere:

- *abilità*, *capacità* di fare qualcosa:
  per questo hanno spesso nomi che terminano in *"-abile"*
  - *confrontabile, serializzabile, clonabile, stampabile…*
  - tipicamente dichiarano *un solo metodo*, il cui nome esprime l'abilità
    - **`compareTo`** per **`Comparable`**, **`print`** per **`Printable`**, etc.
  - a volte, però, non ne dichiarano affatto: *sono vuote!*
    Sembra un assurdo, ma non lo è: *sono marker*
    - «tag» usati per *esplicitare* che una proprietà è *già presente…*
    - … sfruttando il compilatore per i relativi controlli incrociati ☺
- concetti astratti
  - *azione, ascoltatore di eventi, servizio di stampa, …*

IL TEMA DEL CONFRONTO

- Tutti i linguaggi offrono fornite funzioni di utilità o librerie per effettuare *ordinamenti* e *ricerche* in strutture dati
- Obiettivo: implementarle in modo generico e configurabile
- A tal fine, il punto chiave è la capacità di *confrontare* due elementi, di qualunque tipo essi siano
- La «confrontabilità» è una *tipica proprietà* che un oggetto *potrebbe* possedere → situazione ideale da rappresentare con un'opportuna *interfaccia*
  - in Java, Kotlin: `Comparable`
  - in C#: `IComparable`
  - in Scala: `Ordered`

CONFRONTABILITÀ

- Dicendo «confrontabilità» si sottintende solitamente la presenza di un *«criterio naturale»* (*natural ordering*) di confronto, *intrinseco* rispetto alla natura dell'oggetto
  - per stringhe appare «naturale» confrontarle *alfabeticamente*
  - per numeri appare «naturale» confrontarli *secondo il loro valore*
  - per **`Counter`** e **`Frazione`** sarebbe «naturale» confrontarli *secondo il loro valore, coerentemente con quanto fatto nella* **`equals`**
- Non sempre tale criterio esiste: le cose non vanno forzate!
  - per Persona quale sarebbe il criterio «naturale» ..?
    (Per cognome? Per nome? Per età? Un mix di questi..?)

CONFRONTABILITÀ & ALGORITMI

- Strutture dati di oggetti *confrontabili* possono essere ordinate in modo altrettanto «naturale» tramite algoritmi generali e polimorfi
  - array: metodo statico **`Arrays.sort`**
  - liste: metodo **`Collections.sort`**
- In tali strutture dati ordinate si possono altresì effettuare ricerche efficienti tramite algoritmi di ricerca binaria
  - array: metodo statico **`Arrays.binarySearch`**
  - liste: metodo **`Collections.binarySearch`**
- … e molto altro

UN'INTERFACCIA PER LA CONFRONTABILITÀ

- Ipotesi: una classe `T` i cui oggetti sono dotati della capacità di «confrontarsi» *implementa un'apposita interfaccia tipizzata*
  - tipizzata perché deve essere specializzata sul tipo T della classe
  - Java: `Comparable<T>` Java
  - C#: `IComparable<T>` C#
  - Scala: `Ordered[T]` Scala
  - Kotlin: `Comparable<T>` Kotlin
- Essa dichiara un solo metodo, che esprime il confronto:
  - Java: `public int compareTo(T that);` Java
  - C#: `public int CompareTo(T that);` C#
  - Scala: `def compare(other : T) : Int;` Scala
  - Kotlin: `public fun compareTo(other : T):Int;` Kotlin

UN'INTERFACCIA PER LA CONFRONTABILITÀ

- Il metodo che esprime il confronto:
  - Java: `public int compareTo(T that);` Java
  - C#: `public int CompareTo(T that);` C#
  - Scala: `def compare(other : T) : Int;` Scala
  - Kotlin: `public fun compareTo(other : T):Int;` Kotlin
- ha la seguente semantica:
  - **`a.compareTo(b)`** restituisce -1 se **`a<b`**, +1 se **`a>b`**, 0 se uguali
  - NB: la funzione deve garantire un comportamento coerente con **`equals`** & **`hashCode`** nel caso dell'uguaglianza

ESEMPIO: UN `Counter` CONFRONTABILE

- Ad esempio, per essere «confrontabile», un **`Counter`** dovrà implementare l'interfaccia:
  - in Java o Kotlin `Comparable<Counter>` Java Kotlin
  - in C# `IComparable<Counter>` C#
  - in Scala `Ordered[Counter]` Scala
- A tal fine dovrà implementare al suo interno il metodo:
  - Java: `public int compareTo(Counter that){…}` Java
  - C#: `public int CompareTo(Counter that) {…}` C#
  - Scala: `def compareTo(other : Counter) : Int = {…}` Scala
  - Kotlin: `public fun compareTo(other : Counter):Int {…}` Kotlin

UN COUNTER `Comparable`

Java

```
public class Counter
          implements Comparable<Counter> {
   ...

   public int compareTo(Counter that) {
        if (val < that.val) return -1;
        if (val > that.val) return +1;
        /* else */ return 0;
   }
}
```

interfaccia tipizzata

Esprimiamo qui il criterio «naturale» di confronto

NB: le versioni in C# Scala Kotlin sono analoghe

UN COUNTER Comparable

Java

```java
public class Counter implements Comparable<Counter> {
    protected int val;
    public Counter() { val = 1; }
    public Counter(int v) { val = v; }
    public void reset() { val = 0; }
    public void inc()   { val++; }
    public int getValue() { return val;}
    //
    public String toString() { return "Counter di valore "+ val;}
    //
    public boolean equals(Object obj){
     if(!(obj instanceof Counter)) return false;
     return this.val == ((Counter)obj).val;
    }
    public int hashCode() { return val * 31; }
    //
    public int compareTo(Counter that) {
        if (val < that.val) return -1;
        if (val > that.val) return +1;
        /* else */ return 0;
    }
}
```

C#

```csharp
using System;

public class Counter : IComparable<Counter> {
    protected int val;
    public Counter() { val = 1; }
    public Counter(int v) { val = v; }
    public void reset() { val = 0; }
    public void inc()   { val++; }
    public int getValue() { return val;}
    //
    public override string ToString() { return "Counter di valore "+ val;}
    //
    public override int GetHashCode() { return val * 31; }
    public override bool Equals(object obj){
     if(!(obj is Counter)) return false;
     return this.val == ((Counter)obj).val;
    }
    //
    public int CompareTo(Counter other) {
        if (val < other.val) return -1;
        if (val > other.val) return +1;
        /* else */ return 0;
    }
}
```

Scala

```scala
class Counter(private var value : Int = 1 ) extends Ordered[Counter] {
    def reset() : Unit = { value = 0; }
    def inc() : Unit   = { value +=1 ; }
    def getValue() : Int = { return value;  }
    protected def setValue(v:Int) : Unit = { value = v;}

    override def equals(other : Any) : Boolean = {
        return  if(other.isInstanceOf[Counter])  value==other.asInstanceOf[Cc
                else false;
    }

    override def hashCode() : Int = {
       return value * 31;
    }

    override def toString() : String = { return "Counter di valore " + value;

    // non si chiama più compareTo/1, ma compare/1
    override def compare(other: Counter) : Int = {
        if (value < other.value) return -1;
        if (value > other.value) return +1;
        /* else */ return 0;
    }
}
```

Kotlin

```kotlin
open public class Counter(value : Int = 1 ) : Comparable<Counter> {
    private var value : Int
    init{ this.value = value;}
    public fun reset() : Unit { value = 0; }
    public fun inc() : Unit   { value++; }
    public fun getValue() : Int { return value; }
    protected fun setValue(v:Int) : Unit { value = v;}

    public override fun equals(other : Any?) : Boolean {
        return if(other is Counter) value==other.value; else false;
    }

    public override fun hashCode() : Int {
       return value * 31;
    }

    public override fun toString() : String { return "Counter di valore " + value; }

    public override fun compareTo(other: Counter) : Int {
        if (value < other.value) return -1;
        if (value > other.value) return +1;
        /* else */ return 0;
    }
}
```

UN PICCOLO MAIN DI PROVA

Java

```java
public class TestComparable {

    public static void main(String args[]) {
        Counter c1 = new Counter(10);
        Counter c2 = new Counter(10);
        //
        System.out.println("c1 vs c2: " + c1.compareTo(c2)); // dà 0
        System.out.println("c1 == c2: " + c1.equals(c2));    // dà true
        c1.inc();
        System.out.println("c1 vs c2: " + c1.compareTo(c2)); // dà +1
        System.out.println("c2 vs c1: " + c2.compareTo(c1)); // dà -1
        System.out.println("c1 == c2: " + c1.equals(c2));    // dà false
    }
}
```

```
---------- Java Run ----------
c1 vs c2: 0
c1 == c2: true
c1 vs c2: 1
c2 vs c1: -1
```

C#

```csharp
class TestCounterComparable {
    public static void Main(string[] args) {
        Counter c1 = new Counter(10);
        Counter c2 = new Counter(10);

        WriteLine("c1 vs c2: " + c1.CompareTo(c2)); // dà 0
        WriteLine("c1 == c2: " + c1.Equals(c2));    // dà true
        ;
        WriteLine("c1 vs c2: " + c1.CompareTo(c2)); // dà +1
        WriteLine("c2 vs c1: " + c2.CompareTo(c1)); // dà -1
        WriteLine("c1 == c2: " + c1.Equals(c2));    // dà false
```

```
---------- C# Run -------
c1 vs c2: 0
c1 == c2: True
c1 vs c2: 1
c2 vs c1: -1
c1 == c2: False
```

Scala

```scala
object Counter {
 def main(args: Array[String]) = {
    val c1 = new Counter(10);
    val c2 = new Counter(10);
    println("c1 vs c2: " + c1.compare(c2)); // dà 0
    println("c1 == c2: " + c1.equals(c2));  // dà true
    c1.inc();
    println("c1 vs c2: " + c1.compare(c2)); // dà +1
    println("c2 vs c1: " + c2.compare(c1)); // dà -1
    println("c1 == c2: " + c1.equals(c2));  // dà false
 }
}
```

```
---------- Scala run ----------
c1 vs c2: 0
c1 == c2: true
c1 vs c2: 1
c2 vs c1: -1
c1 == c2: false
```

Kotlin

```kotlin
fun main(args: Array<String>){
    val c1 = Counter(10);
    val c2 = Counter(10);
    println("c1 vs c2: " + c1.compareTo(c2)); // dà 0
    println("c1 == c2: " + c1.equals(c2));    // dà true
    c1.inc();
    println("c1 vs c2: " + c1.compareTo(c2)); // dà +1
    println("c2 vs c1: " + c2.compareTo(c1)); // dà -1
    println("c1 == c2: " + c1.equals(c2));    // dà false
}
```

```
---------- Kotlin run ----------
c1 vs c2: 0
c1 == c2: true
c1 vs c2: 1
c2 vs c1: -1
c1 == c2: false
```

ORDINARE UN ARRAY DI COUNTER

Java

```
public class Test {
   public static void main(String args[]) {
    Counter[] myCounterArray = {
       new Counter(110), new Counter(100),
       new Counter(30),  new Counter(50)  };
    for(Counter c: myCounterArray) System.out.println(c);
    Arrays.sort(myCounterArray);
    for(Counter c: myCounterArray) System.out.println(c);
   }
}
```

Internamente, **`sort`** invoca **`v[i].compareTo(v[j])`** per fare i necessari confronti.

```
Counter di valore 110
Counter di valore 100
Counter di valore 30
Counter di valore 50
Counter di valore 30
Counter di valore 50
Counter di valore 100
Counter di valore 110
```

ORDINARE UN ARRAY DI COUNTER

C#

```
class SortArrayOfComparable {
    public static void Main(string[] args) {
        Counter[] myCounterArray = {
            new Counter(110), new Counter(100),
            new Counter(30), new Counter(50) };

        foreach(Counter c in myCounterArray) Console.WriteLine(c);

        Array.Sort(myCounterArray);

        foreach(Counter c in myCounterArray) Console.WriteLine(c);
    }
}
```

Scala

```
object SortArrayOfOrdered {

 def main(args: Array[String]) = {
    val myCounterArray : Array[Counter] = Array(
        new Counter(110), new Counter(100), new Counter(30), new Counter(50) );

    for(c <- myCounterArray) println(c);

    scala.util.Sorting.quickSort(myCounterArray); // the old way
    // altenatively you can use the sorted() method, which produces a new array
    // without altering the original one:
    // for(c <- myCounterArray.sorted) println(c);

    for(c <- myCounterArray) println(c);

 }
}
```

Kotlin

```
fun main(args: Array<String>) : Unit {
    val myCounterArray : Array<Counter> = arrayOf(
        Counter(110), Counter(100), Counter(30), Counter(50) );

    for(c in myCounterArray) println(c);

    myCounterArray.sort();

    for(c in myCounterArray) println(c);
}
```

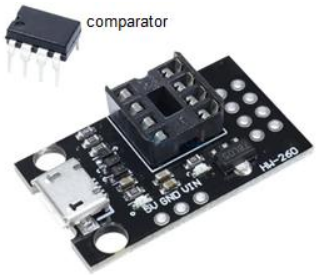
UN ALTRO ESEMPIO: Frazioni confrontabili

- Analogamente, per essere «confrontabile», una **`Frazione`** dovrà implementare l'interfaccia:
  - in Java o Kotlin **`Comparable<Frazione>`** Java Kotlin
  - in C# **`IComparable<Frazione>`** C#
  - in Scala **`Ordered[Frazione]`** Scala
- il cui metodo di confronto dovrà incapsulare un criterio molto simile a quello di «equivalenza» già incapsulato in **`equals`**

Please do it by yourself ☺

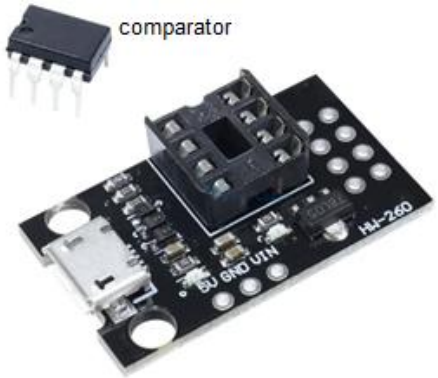
CONFRONTABILITÀ & ALGORITMI

- La confrontabilità secondo un *«criterio naturale»* non cattura però tutte le esigenze, perché:
  - può non esistere un criterio «naturale»
  - anche se esiste, può accadere che non se ne sia tenuto conto in fase di progetto di quella classe
  - può comunque sorgere l'esigenza di confrontare elementi, in alcune situazioni, *con un criterio diverso*
- A questo fine si introduce il concetto di *comparatore*
  - un componente «terzo» che *incapsula un arbitrario criterio di confronto* fra elementi di un certo tipo
  - analogia hardware: un chip «pluggabile», che porta in dote una sua capacità di «confrontare cose»



CONFRONTABILITÀ TRAMITE COMPARATORI

- Un comparatore è l'analogo software di un chip hardware: incorpora una relazione d'ordine
  - ordinamento di **`Counter`** modulo quel-che-vogliamo…
  - … ordinamento di stringhe per lunghezza, o ignorando gli spazi…
  - ... e ogni altra necessità *diversa dall' «ordinamento naturale»*
- È un componente «pluggabile»: dovunque venga inserito, «porta in dote» il criterio di confronto che incapsula
  - in particolare, quindi, basta passare a **`sort`** un certo comparatore per ordinare la struttura dati secondo il criterio del comparatore
  - *indipendentemente* dal fatto che la struttura avesse, o meno, un proprio criterio «naturale» espresso da **`Comparable`**



CONFRONTABILITÀ TRAMITE COMPARATORI: INTERFACCIA

- Un comparatore per **`T`** è una classe che incorpora un metodo che confronta *due argomenti* di tipo T
  - a differenza di **`Comparable`**, non esprime un metodo dell'oggetto da confrontare: il comparatore è un *ente terzo,* che confronta due oggetti
  - Java, Kotlin: `Comparator<T>` — Java, Kotlin
  - C#: `IComparer<T>` — C#
  - Scala: `Ordering[T]` — Scala
- Dichiara un solo metodo, che cattura il criterio di confronto:
  - Java: `public int compare(T a, T b);` — Java
  - C#: `public int Compare(T a, T b);` — C#
  - Scala: `def compare(a:T, b:T) : Int;` — Scala
  - Kotlin: `public fun compare(a:T, b:T) : Int` — Kotlin

CONFRONTABILITÀ TRAMITE COMPARATORI: INTERFACCIA

- Il metodo che esprime il confronto:
  - Java: `public int compare(T a, T b);` Java
  - C#: `public int Compare(T a, T b);` C#
  - Scala: `def compare(a:T, b:T) : Int;` Scala
  - Kotlin: `public fun compare(a:T, b:T) : Int;` Kotlin
- ha una seguente semantica analoga alla precedente:
  - **`cmp.compare(a,b)`** restituisce -1 se **`a<b`**, +1 se **`a>b`**, 0 se uguali
  - **`cmp`** è una istanza di comparatore che incapsula il criterio di confronto desiderato
- Si possono definire *molti comparatori diversi:* idealmente, uno per ogni criterio di confronto che serve ☺

ESEMPIO: UN COMPARATORE PER `Counter`

- Ad esempio, un comparatore per **`Counter`** dovrà implementare l'interfaccia:
  - in Java o Kotlin `Comparator<Counter>` Java Kotlin
  - in C# `IComparer<Counter>` C#
  - in Scala `Ordering[Counter]` Scala
- A tal fine dovrà implementare al suo interno il metodo:
  - Java: `public int compare(Counter a,Counter b){…}` Java
  - C#: `public int Compare(Counter a,Counter b){…}` C#
  - Scala: `def compare(a:Counter,b:Counter):Int = {…}` Scala
  - Kotlin: `public fun compare(a:Counter,b:Counter):Int{…}` Kotlin

ESEMPIO: UN COMPARATORE PER `Counter`

- Supponiamo di voler ordinare un array di **`Counter`** in base al loro valore *modulo 24*
- Non possiamo né vogliamo modificare l'ordinamento naturale dei **`Counter`**, che è crescente per valore (e va bene per tanti altri usi): *vogliamo risolvere una specifica esigenza*
- Implementiamo un comparatore che inglobi questo criterio:

Java

```
public class MyComp implements Comparator<Counter> {
   public int compare(Counter x, Counter y) {
        if (x.val%24 < y.val%24) return -1;
        if (x.val%24 > y.val%24) return +1;
        /* else */ return 0;
   }
}
```

ESEMPIO: UN COMPARATORE PER `Counter`

C#

```
class MyComp : IComparer<Counter> {
    public int Compare(Counter x, Counter y) {
        if (x.getValue()%24 < y.getValue()%24) return -1;
        if (x.getValue()%24 > y.getValue()%24) return +1;
        /* else */ return 0;
    }
}
```

Scala

```
class MyComp extends Ordering[Counter] {
    override def compare(a: Counter, b: Counter) : Int = {
        if (a.getValue()%24 < b.getValue()%24) return -1;
        if (a.getValue()%24 > b.getValue()%24) return +1;
        /* else */ return 0;
    }
}
```

Kotlin

```
class MyComp : Comparator<Counter> {
    public override fun compare(a: Counter, b: Counter) : Int {
        if (a.getValue()%24 < b.getValue()%24) return -1;
        if (a.getValue()%24 > b.getValue()%24) return +1;
        /* else */ return 0;
    }
}
```

ORDINARE UN ARRAY DI COUNTER CON UN COMPARATORE AD HOC

Java

~C#

~Scala

~Kotlin

```
public class Test {
   public static void main(String args[]) {
    Counter[] myCounterArray = {
        new Counter(110), new Counter(100),
        new Counter(30),  new Counter(50)  };
    for(Counter c: myCounterArray) System.out.println(c);
    Arrays.sort(myCounterArray, new MyComp() );
    for(Counter c: myCounterArray) System.out.println(c);
   }
}
```

Usa questo comparatore invece dell'ordinamento *naturale* dei Counter

Perché

50 % 24 = 2 *30 % 24 = 6*

100 % 24 = 4 *110 % 24 = 14*

```
Counter di valore 110
Counter di valore 100
Counter di valore 30
Counter di valore 50
Counter di valore 50
Counter di valore 100
Counter di valore 30
Counter di valore 110
```

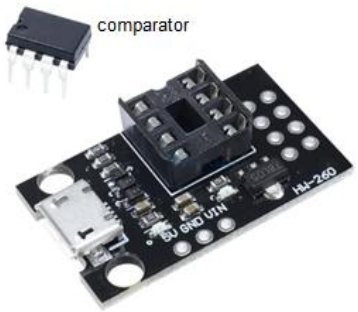
UN ALTRO ESEMPIO: COMPARATORI DI PERSONE

- La classe **`Persona`** sviluppata tempo addietro
  - proprietà: cognome, nome, anno di nascita

  non incorporava alcun criterio di confronto
- Si potrebbe renderla confrontabile implementando **`Comparable`**, ma.. esiste un criterio davvero «naturale»?
- In realtà, no:
  - ci sono pari ragioni per ordinarle per cognome come per nome o età, come pure con una qualunque combinazione di questi, o altri
  - *non c'è un vero criterio «naturale»* come per le frazioni o le stringhe!
- Imporre un criterio «built-in» tramite **`Comparable`** sarebbe quindi una forzatura: molto meglio dei *comparatori esterni*

UN ALTRO ESEMPIO: COMPARATORI DI PERSONE

- Potrebbero servirci comparatori
  - per cognome
  - per nome
  - per anno di nascita (o età)



- Facile: basta fare tre classi-**`Comparator`**, ognuna con una diversa implementazione del metodo **`compare`**
  - poi basterà passare al metodo **`sort`** il «giusto» comparatore ☺

Java

```
Arrays.sort(persone, new CognomeComparator() );
System.out.println(String.join(", ", persone));

Arrays.sort(persone, new NomeComparator() );
System.out.println(String.join(", ", persone));

Arrays.sort(persone, new EtaComparator() );
System.out.println(String.join(", ", persone));
```

UN ALTRO ESEMPIO: COMPARATORI DI PERSONE

Java

```
class CognomeComparator implements Comparator<Persona> {
   public int compare(Persona p1, Persona p2){
    return p1.getCognome().compareTo(p2.getCognome());
   }
}
```

Sfrutta il fatto che le stringhe siano **Comparable** e abbiano quindi una loro **compareTo**

```
class NomeComparator implements Comparator<Persona>{
   public int compare(Persona p1, Persona p2){
    return p1.getNome().compareTo(p2.getNome());
   }
}
```

Sfrutta il fatto che le stringhe siano **Comparable** e abbiano quindi una loro **compareTo**

```
class EtaComparator implements Comparator<Persona>{
   public int compare(Persona p1, Persona p2){
    return Integer.compare(p1.getEtà(), p2.getEtà());
   }
}
```

Sfrutta la funzione statica di confronto per **int** della classe **Integer**

UN ALTRO ESEMPIO: ORDINARE UN ARRAY DI PERSONE

- Ordiniamo l'array nei tre modi e stampiamo:

```
System.out.println(String.join(", ", persone));

Arrays.sort(persone, new CognomeComparator() );
System.out.println(String.join(", ", persone));

Arrays.sort(persone, new NomeComparator() );
System.out.println(String.join(", ", persone));

Arrays.sort(persone, new EtaComparator() );
System.out.println(String.join(", ", persone));
```

Modo furbo per stampare un array di stringhe

Java

```
------- unsorted -------
John Doe ha 25 anni
Jane Doe ha 45 anni
Anne Bee ha 31 anni
Jane Doe ha 22 anni
----- per cognome ------
Anne Bee ha 31 anni
John Doe ha 25 anni
Jane Doe ha 45 anni
Jane Doe ha 22 anni
```

```
------- per nome ------
Anne Bee ha 31 anni
Jane Doe ha 45 anni
Jane Doe ha 22 anni
John Doe ha 25 anni
------- per età -------
Jane Doe ha 22 anni
John Doe ha 25 anni
Anne Bee ha 31 anni
Jane Doe ha 45 anni
```

UN ALTRO ESEMPIO: ORDINARE UN ARRAY DI PERSONE

- Ovviamente, i comparatori si possono definire anche come oggetti a se stanti, eventualmente riusabili più volte:

Java

```
Comparator<Persona> cmp1 = new CognomeComparator();
Comparator<Persona> cmp2 = new NomeComparator();
Comparator<Persona> cmp3 = new EtaComparator();

print("unsorted", persone);

Arrays.sort(persone, cmp1 );
System.out.println(String.join(", ", persone));

Arrays.sort(persone, cmp2 );
System.out.println(String.join(", ", persone));

Arrays.sort(persone, cmp3 );
System.out.println(String.join(", ", persone));
```

Riusabili più e più volte nel programma: tanto, *non hanno stato!*

UN ALTRO ESEMPIO: ORDINARE UN ARRAY DI PERSONE

- Questo approccio funziona, ma:
  - porta a definire *una classe per ogni criterio,* per poi *istanziarla una e una sola volta*
  - se la singola istanza di ogni comparatore è usata una sola volta, appoggiarla a un riferimento esplicito (**`cmp1`**, **`cmp2`**, **`com3`**) è inutile: si può fare la **`new`** direttamente *inline*

Molto chiaro
MA anche, secondo alcuni, *prolisso e verboso*

Java

```
Comparator<Persona> cmp1 = new CognomeComparator();
Comparator<Persona> cmp2 = new NomeComparator();
Comparator<Persona> cmp3 = new EtaComparator();

Arrays.sort(persone, cmp1 );
print("per solo cognome", persone);

Arrays.sort(persone, cmp2 );
print("per solo nome", persone);

Arrays.sort(persone, cmp3 );
print("per sola età", persone);
```

IMPLEMENTAZIONE «INLINE» CON CLASSI ANONIME

- Se un comparatore serve *una sola volta* e ne viene costruita *una sola istanza,* un'alternativa è evitare di scrivere la classe esplicitamente, creando un'istanza di *CLASSE ANONIMA*
  - si evita di scrivere direttamente una nuova classe perché *se ne delega la creazione al compilatore, «sotto banco»*
  - si collassano tre cose insieme:
    - la definizione della classe-comparatore
    - la definizione al suo interno della **`compare`** appropriata
    - la costruzione dell'istanza-singleton del comparatore

Vediamo come

IMPLEMENTAZIONE «INLINE» CON CLASSI ANONIME

- Finora, per usare un nostro comparatore, abbiamo dovuto:
  - definire una classe-comparatore specifica

```
class CognomeComparator implements Comparator<Persona> {
    ...
}
```

Java

  - scriverci dentro la **`compare`** appropriata

```
class CognomeComparator implements Comparator<Persona> {
    public int compare(Persona p1, Persona p2){
     return p1.getCognome().compareTo(p2.getCognome());
    }
}
```

Java

  - creare (e passare a **`sort`**) l'istanza-singleton di *quel* comparatore:

```
Arrays.sort(persone, new CognomeComparator() );
```

Java

IMPLEMENTAZIONE «INLINE» CON CLASSI ANONIME

- Con una classe anonima, i tre passaggi collassano:
  - NON si inventa più il nome della classe che implementa **`Comparator`**, perché *lo farà il compilatore al posto nostro*

```
class CognomeComparator implements Comparator<Persona> {
   public int compare(Persona p1, Persona p2){
    return p1.getCognome().compareTo(p2.getCognome());
   }
}
```

Java

  - di conseguenza, anziché costruire il comparatore «per nome»:

```
Arrays.sort(persone, new CognomeComparator() );
```

Java

  - si scrive direttamente la **`new`** di «tutto quel che resta", *inline*:

```
Arrays.sort(persone, new Comparator() { … } );
```

Codice di **`compare`**

Java

IMPLEMENTAZIONE «INLINE» CON CLASSI ANONIME

- Risultato: un'implementazione *anonima!*

```
Arrays.sort(persone, new Comparator<Persona>() {
   public int compare(Persona p1, Persona p2){
    return p1.getCognome().compareTo(p2.getCognome());
   }
 );
```

Java

  - ATTENZIONE! Non stiamo tentando di "istanziare un'interfaccia" (cosa vietata e comunque assurda, sarebbe un guscio vuoto): stiamo semplicemente *lasciando scegliere al compilatore il nome della classe* che implementa tale interfaccia (classe che comunque *deve* esistere!)
  - infatti, alla fine ci troveremo una classe in più nel bytecode, di nome *ClasseEsterna$1.class*

ORDINARE UN ARRAY DI PERSONE CON CLASSE ANONIMA

Java

```
public class TestPersone {
   public static void main(String args[]) {
     ...
     Arrays.sort(persone, new CognomeComparator() );
   }
}
```

PRIMA

DOPO

```
public class TestPersone {
   public static void main(String args[]) {
     ...
     Arrays.sort(persone,
        new Comparator<Persona>() {
          public int compare(Persona p1, Persona p2){...}
        }
     );
   }
}
```

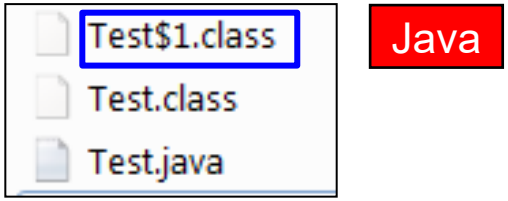
Questo box era **CognomeComparator**

Non istanzia l'interfaccia: istanzia la classe anonima creata per noi dal compilatore «dietro le quinte»

Implementazione inline tramite CLASSE ANONIMA

E INFATTI, DIETRO LE QUINTE…

- Dietro le quinte, il compilatore genera effettivamente una nuova classe, con nome uguale alla ***classe ospite+$1***
  - basta guardare i file prodotti!
  - se la classe esterna si chiama **`Test`**, i due file **`.class`** prodotti si chiamano **`Test`** e **`Test$1`**



- Conclusione: fu vera gloria?
  - di per sé, le classi anonime non sembrano un gran guadagno…
  - in effetti, il loro vero valore sta nel passo logico successivo: le *lambda expression* (che vedremo presto)

COMPARATORI ANONIMI NEGLI ALTRI LINGUAGGI

- Le classi anonime esistono anche in C#, Scala, Koltin
- In Scala e Kotlin, la nozione di *oggetto singleton* incorporata nel linguaggio (keyword **`object`**) snellisce anche la sintassi
  - in Scala, anziché scrivere **`new Ordering[Tipo]{…}`**:

```
scala.util.Sorting.quickSort(myCounterArray)(
  new Ordering[Counter] {
    def compare(a: Counter, b: Counter) :
})
```

  - si può passare o definire al volo un *object singleton*: →

```
    scala.util.Sorting.quickSort(myCounterArray)(MyComp)
    // using an object, no need to instantiate a new Comparator
    // otherwise a class could be used, with the new keyword here

    scala.util.Sorting.quickSort(myCounterArray)(MyBetterComp)
    // using an object, no need to instantiate a new Comparator
    // otherwise a class could be used, with the new keyword here

 }
}

object MyComp extends Ordering[Counter] {
    override def compare(a: Counter, b: Counter) : Int = {
        if (a.getValue()%24 < b.getValue()%24) return -1;
        if (a.getValue()%24 > b.getValue()%24) return +1;
        /* else */ return 0;
    }
}

object MyBetterComp extends Ordering[Counter] {
    override def compare(a: Counter, b: Counter) : Int =
        a.getValue()%24 compare b.getValue()%24;
}
```

Scala

COMPARATORI ANONIMI NEGLI ALTRI LINGUAGGI

- Le classi anonime esistono anche in C#, Scala, Koltin
- In Scala e Kotlin, la nozione di *oggetto singleton* incorporata nel linguaggio (keyword **`object`**) snellisce anche la sintassi
  - analogamente, in Kotlin conviene *creare inline il singleton stesso* con la notazione **`object:Comparator<Tipo> {…}`**

```
myCounterArray.sortWith( object: Comparator<Counter> {
  override fun compare(a: Counter, b: Counter) : Int {
    if (a.getValue()%24 < b.getValue()%24) return -1;
    if (a.getValue()%24 > b.getValue()%24) return +1;
    /* else */ return 0;
  }
});
```

Kotlin

Interfacce marker

IL LIMITE ESTREMO: INTERFACCE VUOTE ("MARKER")

- Per sfruttare appieno le interfacce come strumento concettuale di modellazione, si introducono talora delle interfacce vuote, dette *marcatori* (*marker*)
- Sono utili proprio *perché introducono un tipo:*
  - implementarle non costa nulla *(non c'è codice da scrivere)*
  - ma *definiscono un tipo* con cui spaziare nella tassonomia di classi
- Ciò permette di sfruttare il compilatore per scovare incon-gruenze nelle scelte e nell'uso dei tipi
  - se una funzione si aspetta un argomento di un certo tipo (interfaccia) e viene passato un oggetto che non lo è, *ci viene segnalato* ☺
  - ESEMPI: *Cloneable, Serializable, …*

INTERFACCE "MARKER": ESEMPIO

- Si supponga che in un sistema *solo certe classi* debbano essere *mostrabili a video* tramite un certo metodo **`show`**
  - naturalmente, ogni classe ha una **`toString`** appropriata, ma noi vogliamo che solo alcune, quelle «taggate", siano *visualizzabili*
- IDEA: introduciamo il tipo-marker **`Dentable`**
  - ASSUNTO: solo le classi (ad-)"dentabili" ☺ devono poter essere visualizzabili

```
public interface Dentable {}
```

Java | C# | ~Scala | Kotlin

  - è un'interfaccia vuota, perché *in realtà non occorre scrivere codice:* tutti gli oggetti hanno già una **`toString`** !
  - lo scopo è *distinguere queste classi dalle altre:* così, possiamo sfruttare il compilatore per garantire che *solo loro si stampino.*

INTERFACCE "MARKER": ESEMPIO

- Definiamo allora il metodo **`show`** in modo che *accetti solo oggetti di tipo* **`Dentable`**

```
public static void show(Dentable d){
   System.out.println(d);
}
```

Anche se concretamente qualunque oggetto avrebbe una **`toString`**…

..grazie al tipo-interfaccia usato come marcatore, solo i *dentabili* si stampano!

Java | ~C# | ~Kotlin | ~Scala

- Risultato: ora è impossibile chiamare **`show`** su oggetti «non **`Dentable`**» ☺
  - il bello è che è il compilatore a fare questo controllo per noi !
  - GARANZIA DI CONSISTENZA: *esprimendo il vincolo attraverso il type system*, abbiamo potuto sfruttare il sistema di type check del compilatore per farci fare questo controllo *gratis* ☺
    RICORDA: il type system è un alleato, non un nemico!

INTERFACCE "MARKER": ESEMPIO

- E infatti…

```
public class Good implements Dentable {
  ...
}
```

Java | ~C# | ~Kotlin | ~Scala

```
public class Bad { // not Dentable
  ...
}
```

Java | ~C# | ~Kotlin | ~Scala

```
Good g = new Good(..);
show(g); // OK

Bad  b = new Bad(..);
show(b); // NO!
```

```
ERROR: method show cannot be applied to
given types
required: Dentable;  found: Bad
Reason: actual argument Bad cannot be
converted to Dentable by method
invocation conversion
```